CISC 322 Assignment 2

A2 Concrete Architecture Report

March 2026

Authors:

Jacob Badali
20jasb1@queensu.ca

Zhu Yuan
18zy11@queensu.ca

Chloe Atherton
20cga3@queensu.ca

Will Pritchard
21whp2@queensu.ca

Julian Tiqui
21jtt4@queensu.ca

Connor Rewa
20car8@queensu.ca

Abstract

Gemini CLI is a developer-focused command-line tool that integrates Google's Gemini models with concrete execution capabilities across local and external developer environments. This report presents a comprehensive concrete architectural analysis of the system, transitioning from its initial conceptual design to its physical implementation within the monorepo. By utilizing the Understand static analysis tool to extract module dependencies, we derive a granular four-tier architecture, encompassing the Presentation, Application, Service, and Infrastructure layers, organized into seven distinct components. The analysis reveals that the concrete implementation strictly decouples orchestration logic from security policies, global configurability, and distributed execution, ensuring isolated policy enforcement across the system. Furthermore, through a detailed reflexion analysis and a second-level deep dive into the Tool Execution Framework, this report highlights how the physical boundaries successfully prevent architectural decay and maintain a strict unidirectional data flow. By tracing concrete use cases such as command debugging and code refactoring, this report illustrates how the system's event-driven schedulers orchestrate real-world developer workflows, demonstrating how the granular physical decomposition balances extensibility with secure task delegation at the code level.

Contents

1 Introduction and Overview **1**

2 Derivation Process **1**

3 Architecture **1**
3.1 Conceptual Architecture Revisions . . . 1
3.2 Concrete Architecture Overview . . . 2
3.3 Concrete Subsystem Decomposition . . . 3
3.3.1 Presentation Layer . . . 3
3.3.2 Application Layer . . . 3
3.3.3 Service Layer . . . 4
3.3.4 Infrastructure Layer . . . 4
3.4 Reflexion Analysis of High-Level Architecture . . . 4
3.5 Detailed Inner Architecture: Tool Execution Framework . . . 5
3.5.1 Conceptual Second-Level Decomposition . . . 5
3.5.2 Concrete Second-Level Decomposition . . . 6
3.5.3 Internal Interactions . . . 6
3.5.4 Design Patterns and Architectural Style . . . 6
3.5.5 Reflexion Analysis of the Tool Execution Framework . . . 6
3.5.6 Summary of Tool Execution Component . . . 7

4 External Interfaces **7**
4.1 User Command Interface . . . 7
4.2 Workspace and Local Execution Interface . . . 8
4.3 Cloud Model Service Interface . . . 8
4.4 MCP External Tool Interface . . . 8
4.5 Agent Delegation Interface . . . 9
4.6 Authentication and Configuration Interface . . . 9

5 Use Cases **9**
5.1 UC1: Debug a Failing Command . . . 9
5.2 UC2: Refactor Code . . . 11

6 Data Dictionary **13**

7 Naming Conventions **13**

8 Conclusions **14**

9 Lessons Learned **14**

10 AI Collaboration Report **15**

1 Introduction and Overview

Gemini CLI is a developer-focused command-line tool that integrates Google's Gemini models with concrete execution capabilities across both local and external developer environments. While our A1 report modeled this system conceptually as a three-tier architecture encompassing the CLI, Core, and Tools subsystems, this report transitions from theoretical design to the physical realization of the system. This Assignment 2 (A2) report presents a concrete architectural analysis of the Gemini CLI.

By utilizing the Scitools Understand static analysis tool to examine module dependencies within the project's monorepo, we derived the system's true structural boundaries as implemented in the source code. Our concrete analysis reveals that the implemented codebase significantly expands upon the initial conceptual design. Rather than three broad subsystems, the physical architecture fragments into a highly granular four-tier layered structure encompassing the Presentation, Application, Service, and Infrastructure layers. Within these layers, the system is organized into seven major components: the Terminal Interface, Orchestration Engine, Tool Execution Framework, External Integration Layer, Task Delegation System, Security and Execution Environment, and Configuration System. This physical decomposition demonstrates a strict decoupling of orchestration logic from security policies, global configurability, and distributed execution constraints.

The remainder of this report details the derivation process and provides a comprehensive breakdown of the concrete subsystems. It features a reflexion analysis comparing our A1 conceptual model against the A2 concrete implementation, highlighting how the physical boundaries successfully prevent architectural decay. The report conducts a detailed second-level analysis of the Tool Execution Framework, outlines the external interfaces, and illustrates the concrete orchestration of real-world developer workflows through representative system use cases. Ultimately, this analysis demonstrates how the system's granular physical decomposition balances extensibility with secure task delegation at the code level.

2 Derivation Process

To derive the concrete architecture of the Gemini CLI, our group systematically transitioned from the conceptual model developed in Assignment 1 to a physically realized model based on the source code. The process began with deploying the Scitools Understand static analysis tool against the Gemini CLI monorepo. Initially, we loaded the packages/cli and packages/core directories into Understand to extract a comprehensive dependency graph of all modules and files. We then mapped our initial A1 conceptual subsystems (CLI, Core, and Tools) against the physical file structure. By iteratively grouping dependent files and analyzing cross-module references, we identified clear structural boundaries that were not apparent in the conceptual phase. For example, while A1 unified all internal and external capabilities into a single "Tools Subsystem," the Understand dependency graph revealed distinct, decoupled physical clusters for local tool execution, external Model Context Protocol (MCP) integrations, and complex Agent-to-Agent (A2A) task delegation. Through this iterative assignment of code files and folders to our A1 components, we refined the architecture into the seven concrete components presented in this report. Finally, using Understand's visualization features, we generated box-and-arrow dependency diagrams to validate that our newly defined sub-systems strictly adhered to a four-tier layered architecture, ensuring the physical implementation aligned with our structural analysis.

3 Architecture

3.1 Conceptual Architecture Revisions

Our A1 report modeled the Gemini CLI system using a three-tier conceptual architecture consisting of the CLI subsystem, Core subsystem, and Tools subsystem.

After reviewing the published conceptual architecture [17], the revised model expands this design into seven major components organized across four architectural layers. The mapping between our A1 conceptual architecture and the revised conceptual architecture, including the responsibilities of each component, is summarized in Table 1 below.

A1 Original Architecture Component	Responsibility	Revised Architecture Component
CLI Subsystem	User interaction, terminal input/output	Terminal Interface
Core Subsystem	Request orchestration, API communication	Orchestration Engine
Tools Subsystem	Internal tool execution	Tool Execution Framework
Tools Subsystem	External tool Integration	External Integration Layer
Core + Tools	Complex task delegation	Task Delegation System
Core Subsystem	Safety policies and sandboxing	Security & Execution Environment
Core Subsystem	Configuration management	Configuration System

Table 1: Mapping of A1 Architecture to Revised Architecture

An image outlining the new concrete architecture can be seen in Figure 1, below.

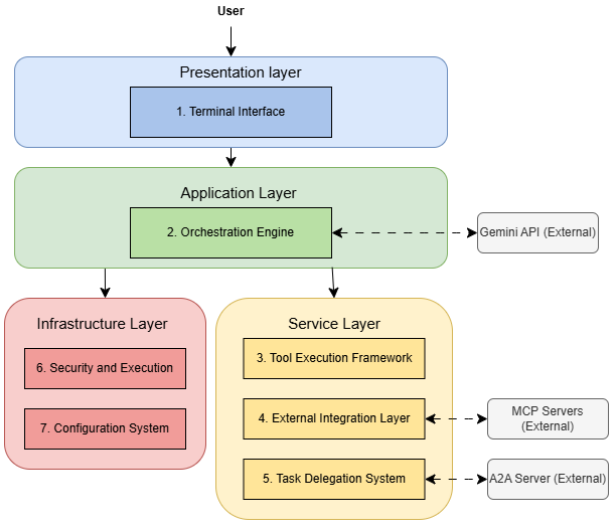

Figure 1: New conceptual architecture outline

The revision does not fundamentally change the architectural interpretation of the system. Instead, it refines the level of abstraction, decomposing several responsibilities that were previously grouped together into more precise components. Aligning our conceptual architecture with the published solution clarifies the system's intended structure and responsibilities, allowing us to more effectively analyze how these components are implemented in the concrete architecture.

3.2 Concrete Architecture Overview

The concrete architecture was derived using the Understand static analysis tool to examine module dependencies within the Gemini CLI repository. The analysis revealed that the majority of system functionality resides within the packages directory, particularly the packages/cli and packages/core subsystems, which aligns with existing high-level architectural analyses of the tool [5, 6]. The CLI package handles terminal interaction and command parsing, while the Core subsystem contains the orchestration engine responsible for routing requests, scheduling tasks, executing tools, and enforcing safety policies. Within the core subsystem, several internal modules further decompose orchestration responsibilities, including routing, scheduling, tool execution, and safety policy enforcement. These modules are described in the following subsystem decomposition.

3.3 Concrete Subsystem Decomposition

The concrete architecture of the Gemini CLI system is realized across a monorepo structure, primarily divided into the packages/cli and packages/core directories. By extracting dependencies using the Understand tool, we refined our initial conceptual model into seven concrete components organized into four architectural layers. Each subsystem encapsulates specific modules and files that handle discrete system responsibilities. The breakdown given from Undestand can be found in Figure 2, below.

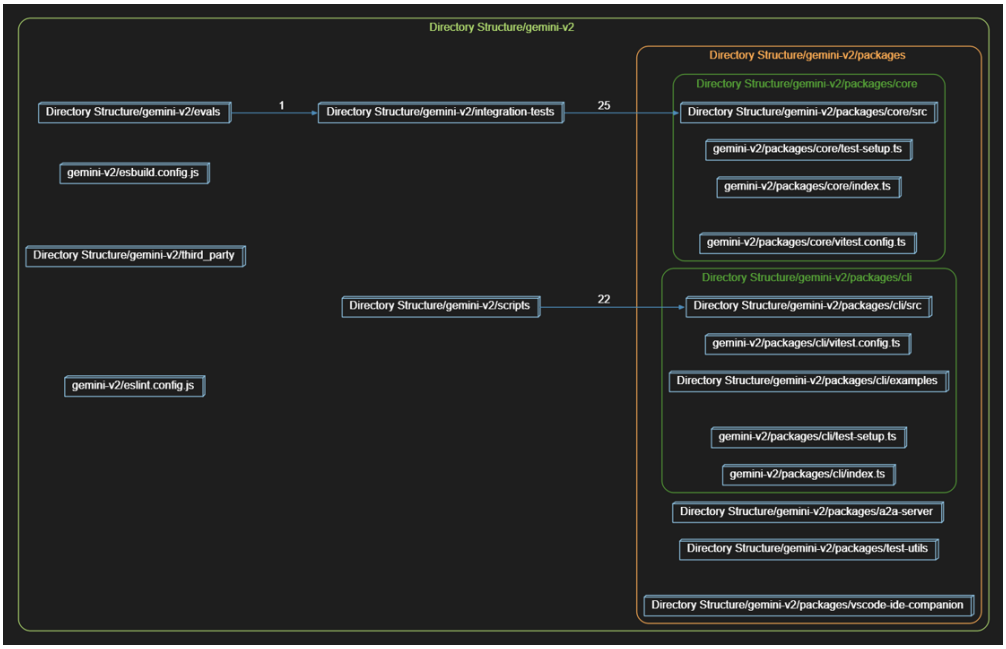

Figure 2: Understand concrete breakdown

3.3.1 Presentation Layer

Component 1: Terminal Interface
The Terminal Interface packages/cli/src/ui serves as the user-facing boundary of the application, responsible for input capture and output rendering. Instead of traditional standard I/O parsing, the concrete implementation leverages React and the Ink library (AppContainer.tsx, gemini.tsx) to render a dynamic, terminal-based DOM [2, 3]. It delegates all business logic to the Application layer via custom hooks (e.g., useGeminiStream.ts).

3.3.2 Application Layer

Component 2: Orchestration Engine
Located at packages/core/src/core, this subsystem acts as the central brain of the CLI. The concrete implementation reveals it is coordinated through several key modules:

- Session & Turn Management (client.ts, turn.ts): Manages the lifecycle of a user request, converting raw text into structured conversational turns.
- LLM Communication (geminiChat.ts, contentGenerator.ts): Handles physical network communication with the Google Gemini API [1], including streaming responses and handling network retries.
- Execution Scheduling (coreToolScheduler.ts): An event-driven coordinator that receives function call requests from the LLM and schedules them for execution in the Service Layer, utilizing asynchronous operational patterns Native to Node.js [4].

3.3.3 Service Layer

Component 3: Tool Execution Framework
Located in packages/core/src/tools, the Tool Execution Framework provides the concrete capabilities the LLM can invoke. It is fundamentally built on a registry pattern (tool-registry.ts), which dynamically loads tools at runtime. Concrete tool implementations include file system operations (read-file.ts, write-file.ts, edit.ts), shell execution (shell.ts), and code search (ripGrep.ts). The framework communicates asynchronously with the Orchestration Engine.

Component 4: External Integration Layer
Located at packages/core/src/tools/mcp-*, contained primarily within mcp-client.ts and mcp-client-manager.ts, this subsystem bridges the Gemini CLI to external tools via the Model Context Protocol (MCP). It acts as a dynamic client that discovers and registers external capabilities into the local Tool Registry, treating external services as native tools.

Component 5: Task Delegation System
Inside of packages/core/src/agents & packages/a2a-server, this subsystem manages complex, multi-step operations that require specialized agentic behaviour. It utilizes agent-scheduler.ts and local-executor.ts to spin up scoped sub-agents (e.g., codebase-investigator.ts). The Orchestration Engine delegates entire complex turns to these agents, which recursively utilize the Tool Execution Framework until a sub-task is complete. Crucially, the inclusion of the a2a-server module enables Agent-to-Agent (A2A) delegation. This allows the local CLI process to coordinate with remote agent instances over HTTP (using protocols defined in packages/a2a-server/src), enabling distributed execution of tasks that may require specialized or heavier isolated compute environments beyond the local CLI constraints.

3.3.4 Infrastructure Layer

Component 6: Security & Execution Environment
Located at packages/core/src/policy, the Security & Execution Environment is isolated into a dedicated subsystem to ensure policies cannot be bypassed by the execution framework. It is driven by the policy-engine.ts, which evaluates requested tool invocations against strict safety rules (e.g., shell-safety.ts). Before any state-modifying tool is executed, the Orchestration Engine must request validation from this layer.

Component 7: Configuration System
Found at packages/core/src/config, the configuration subsystem handles environment setup, authentication keys, and user preferences. Implemented via config.ts and toml-loader.ts, this module parses local configuration files and injects context into both the Orchestration Engine and the Security Environment. Extracted dependency data shows that almost every other subsystem holds a dependency on the Configuration System to retrieve runtime settings.

3.4 Reflexion Analysis of High-Level Architecture

Based on our 3-tier conceptual model proposed in Assignment 1 (which identified the CLI, Core, and Tools subsystems), our static dependency analysis of the Gemini CLI reveals a significantly more granular concrete architecture. The implemented codebase diverges from our conceptual model primarily through the deep breakdown of the "Core" and "Tools" subsystems to handle security, configurability, and distributed execution.

The most notable difference is the fragmentation of the conceptual Core Subsystem. In A1, we envisioned a monolithic Core handling orchestration, security, and settings. However, our concrete analysis proves that the implementation strictly decouples these responsibilities into three distinct architectural components across two layers: the Orchestration Engine, the Security & Execution Environment, and the Configuration System. The rationale for this physical separation in the codebase is separation of concerns; security policies must be able to strictly wrap and veto tool execution independently of the orchestration routing logic. Furthermore, the Configuration System must be globally accessible to nearly every module in the application (such as the config.ts module), a dependency pattern that a monolithic Core would not allow for.

Secondly, our conceptual model aggregated all external and internal capabilities into a single Tools Subsystem.

The concrete architecture diverges by physically splitting this into the Tool Execution Framework, the External Integration Layer, and the Task Delegation System. The rationale for this structural divergence is the difference in lifecycle and security requirements. Internal filesystem and shell operations run locally, whereas the concrete implementation requires dedicated, complex modules to broker connections to external (MCP) servers or A2A remote instances over HTTP. Wrapping them into one conceptual block failed to capture the necessity of these distinct network boundaries.

Finally, while parts of the conceptual architecture were rewritten, static analysis proves our original intuition regarding system layering was successfully preserved. As illustrated in the difference between our A1 conceptual diagram and the A2 concrete dependency diagram, the Presentation Layer (packages/cli/src/ui) maintains a strict unidirectional boundary. It never directly accesses the Execution/Service or Infrastructure layers, depending solely on the Application Layer. This confirms our conceptual intent and demonstrates that the physical implementation avoided architectural decay or "bypasses" across the system's foundational boundaries.

3.5 Detailed Inner Architecture: Tool Execution Framework

The Tool Execution Framework was chosen as a subsystem for a deep dive due to its importance as a boundary between model-driven reasoning and actual actions executed in the developer's local environment. In conceptual architecture, tools are represented as a single execution-oriented subsystem below the Core [7]. In concrete architecture, it is revealed that this is a more intricate collaboration of tool discovery, scheduling, policy enforcement, confirmation management, execution, and result propagation [8, 9, 10, 11, 12, 13, 14]. This makes it a suitable subsystem for a second-level analysis due to its intrinsic complexity that exposes a structure while at the same time being sufficiently bounded to allow tracing of behaviour.

A diagram of the detailed inner architecture of the Tool Executor it's sub components is shown below.

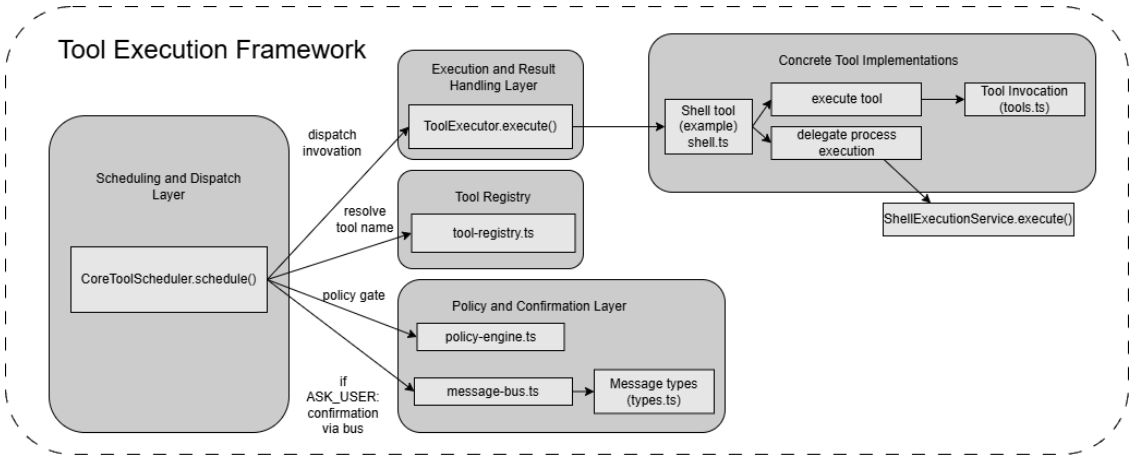

Figure 3: Detailed inner architecture of the Tool Executor Component

3.5.1 Conceptual Second-Level Decomposition

At the conceptual second level, the Tool Execution Framework can be decomposed into five sub-components. First, the Tool Registry is responsible for keeping track of available tools and mapping tool names to actual tool implementations [8]. Second, the Scheduling and Dispatch Layer is responsible for receiving tool call requests and scheduling when these requests are validated and executed [9]. Third, the Policy and Confirmation Layer is responsible for validating whether a requested action is allowed, denied, or needs explicit confirmation [11, 13, 14]. Fourth, the Concrete Tool Implementations are responsible for implementing the capabilities that are exposed to the model, such as shell execution, file access, or code search [8, 11]. Finally, the Execution and Result Handling Layer is responsible for executing the invocation and returning it as a result that can be supplied back to the model [10]. This decomposition of the Tool Execution Framework further refines the A1

conceptual view of the system, where the Tools Subsystem was treated as a single conceptual entity under Core coordination [7].

3.5.2 Concrete Second-Level Decomposition

The conceptual decomposition maps clearly to the concrete source files. The Tool Registry is implemented by tool-registry.ts, which manages tool registration, lookup, sorting, declaration generation, and discovery of external command-based tools [8]. The Scheduling and Dispatch Layer is implemented by coreToolScheduler.ts, which manages tool-call state, queues, approvals, cancellations, and batch completion [9]. The Execution Layer is implemented by tool-executor.ts, which performs the invocation and converts results into structured function responses [10]. The Policy and Confirmation Layer is implemented through policy-engine.ts and message-bus.ts, which evaluate tool-call requests and coordinate confirmation requests and responses [13, 14]. Finally, the Concrete Tool Layer includes implementations such as shell.ts, while lower-level platform-facing behaviour is delegated to shellExecutionService.ts [11, 12]. Collectively, these files indicate that the subsystem is concretely layered rather than physically realized as a single flat module [16].

3.5.3 Internal Interactions

The internal interaction of this subsystem can be traced as a staged execution pipeline. A tool call first enters CoreToolScheduler.schedule(...), where the scheduler creates and manages the tool call object and resolves the requested tool using the configured registry [8, 9]. Before execution occurs, the scheduler validates the request through PolicyEngine.check(...) [9, 13]. If the action is denied, the scheduler returns an error result immediately [9]. If it is allowed, the request advances toward execution. If confirmation is required, the scheduler and tool flow coordinate through the message bus so that approval can be obtained before proceeding [9, 14]. Once approved, ToolExecutor.execute(...) invokes the concrete tool implementation and returns a completed result to the upper layers [10]. This interaction shows that tool execution is not a direct one-step call, but rather a mediated flow with explicit validation, approval, execution, and completion phases [9, 10, 14].

The shell tool provides the clearest example of this flow. In the concrete implementation, ShellToolInvocation prepares a description of the command, derives confirmation metadata, and delegates actual process execution to ShellExecutionService.execute(...) [11]. The shell execution service manages low-level concerns such as PTY-based execution, fallback to child_process, streaming output, process lifecycle handling, and cancellation [12]. This separation indicates that shell behaviour is considered sufficiently complex to justify an additional service boundary beneath the higher-level tool interface [11, 12].

3.5.4 Design Patterns and Architectural Style

Several design patterns are visible in this subsystem. The most prominent is the Registry pattern, implemented by ToolRegistry, which decouples tool users from concrete tool construction and supports extensibility through dynamic tool registration [8]. The subsystem also exhibits a Command-style structure, since each tool invocation encapsulates executable behaviour behind a common invocation interface [10, 11]. The interaction between PolicyEngine, MessageBus, and the scheduler resembles an event-driven publish-subscribe style, since approval requests and responses are exchanged through typed messages rather than tightly bound direct calls [9, 13, 14, 16]. Finally, the split between ShellToolInvocation and ShellExecutionService resembles a Facade or Service Wrapper approach, in which the higher-level tool interface hides lower-level process management and platform-specific execution details [11, 12]. These patterns reinforce the broader layered and event-driven characteristics already identified in the higher-level architecture [7, 16].

3.5.5 Reflexion Analysis of the Tool Execution Framework

The second-level reflexion reveals that the concrete architecture is more layered than the conceptual architecture proposed in A1 [7]. In the conceptual model, the Tools Subsystem was treated as a single execution component coordinated by the Core [7]. In the concrete implementation, tool execution is divided across several cooperating modules, including a registry, a scheduler, a policy engine, a message bus, an executor, and specialized tool handlers [8, 9, 10, 11, 12, 13, 14]. As a result, what appeared conceptually as one subsystem

is implemented as a collaboration of multiple internally distinct services. A likely rationale for this structure is separation of concerns: policy logic remains independent of orchestration details so that tools can evolve without requiring changes to the scheduling layer [9, 10, 11, 12, 13, 14].

A second divergence is that the conceptual architecture implied a relatively uniform abstraction for all tools, whereas the concrete implementation gives shell execution substantially more structural weight than simpler tools [7, 11, 12]. Shell execution includes confirmation detail generation, policy-sensitive command parsing, streaming output, timeouts, cancellation handling, and dedicated process execution [11, 12, 13]. This suggests that the implementation recognizes shell commands as a higher-risk and higher-complexity category than passive inspection operations such as reading files. From a reflexion-model perspective, this is a justified refinement rather than architectural decay: the concrete architecture elaborates the conceptual Tools Subsystem to satisfy safety, interactivity, and platform-specific execution concerns that were abstracted away at the conceptual level [15]. The strongest form of this analysis would additionally include version-history evidence identifying when these internal layers were introduced and why [15].

3.5.6 Summary of Tool Execution Component

Overall, the Tool Execution Framework presents a layered, safety-aware, and extensible internal architecture. It preserves the conceptual role of the Tools Subsystem from A1 but refines that role into a concrete collaboration of registry, scheduling, policy, confirmation, execution, and specialized handlers. The elaboration of this subsystem is beneficial for extensibility, as new tools can be incorporated by registration, as well as for safety, as operations that could have a harmful effect are mediated by policy and confirmation prior to interacting with the local system. As such, this subsystem represents a compelling example of how Gemini CLI's concrete architecture elaborates upon the original conceptual design.

4 External Interfaces

The Gemini CLI's external environment consists of many different actors. The Gemini CLI interacts with users, local development environments, cloud model services, other external tools, remote agents, and authentication or configuration services. Each of these actors will have a dedicated interface within the Gemini CLI's concrete architecture via specific components that are relevant to the implemented components of the system (for example, the Terminal Interface, Orchestration Engine, Tool Execution Framework, External Integration Layer, Task Delegation System, and Configuration System). The section on External Interfaces should address not only the information exchanged at system boundaries, but also which specific subsystem of the implemented code base is responsible for performing the exchange.

4.1 User Command Interface

In the Gemini's User Command Interface (UCI) found inside the Gemini CLI is the boundary between a user and the system. The user interacts with the system through the terminal by entering command requests and code-related prompts or instructions, and in turn, the system processes the user request(s) and returns a response. This UCI is implemented in the Presentation Layer, using the Terminal Interface component to interface with the user, which can be found in packages/cli/src/ui. The UCI uses React and Ink to provide the terminal-based UI, and modules such as AppContainer.tsx, gemini.tsx and useGeminiStream.tsx support the interaction flow. The UCI provides multiple entry points into the UCI and supports the interaction flow using multiple modules including, AppContainer.tsx,gemini.tsx, and useGeminiStream.tsx. Information exchanged across the boundary between UCI and the System is primarily a textual-based format. The User provides the System with prompted commands, corrects input, and generates responses, execution results, summaries, and/or errors. Therefore user command interface is not only the conceptual CLI boundary; it is also the concrete Presentation Sub-System that takes user-generated input and feeds it to the rest of the system for orchestration and execution. Architecturally, the User Command Interface provides a means for the User Command Interface to serve as the concrete entry point for the System and provide the separation of Terminal interaction from Main System Execution Logic, which contributes to an organized distinction between Presentation Responsibilities.

4.2 Workspace and Local Execution Interface

The Local Workspace and Execution Interface is the exchange of data between the Gemini CLI and user's local project environment. This interface serves as the boundary for the system to read and write files, execute commands, and read or modify the file system in the user's local environment. Within the concrete architecture, this interface is primarily implemented by the Tool Execution Framework, Security, and Execution Environment. For example, this interface is supported by concrete tool modules for reading, writing, editing, searching, and shell execution, such as read-file.ts, write-file.ts, edit.ts, and shell.ts. The system can read from the local project environment file system via this channel to obtain file contents, search results, command output, local project-related information, etc. Additionally, the system may also write files, make edits to files, and execute shell commands on behalf of the user to fulfill user request(s). As a result, the information exchanged between the two parties contains not only passive, context-related data but also executables and results of state change. In other words, this interface allows the system and user to directly communicate regarding the user's local workspace rather than simply producing suggestive output. The importance of the boundary to the architectural view is that the user's local environment is accessed via the execution subsystem, and not directly through the user-facing layer. In addition, the execution subsystem is secure and compliant with security and policy mechanisms before performing potentially harmful actions to the user's local environment.

4.3 Cloud Model Service Interface

Gemini Command Line Interface serves as the medium for establishing a connection between the local interactive system (i.e., Gemini CLI) and the remote model evidence-based models supplied to the Cloud Model Service. This middleware layer is controlled by the Orchestration Engine, which interconnects the local interactive system with the cloud-based infrastructure to perform the function of interacting with a remote inference service based on an execution workflow. The local system transmits user prompts, context data, execution status, and tool-related information to the cloud-based model service via this cloud service interface. In return, the cloud service transmits back either final (completed) responses generated by the model service, streamed content of generated data from the cloud service, and/or requests for function/call (Forwarded Request for Execution) to begin additional actions in the local system. Therefore, the output received from the cloud service could be used either as a final response or as intermediate results for executing subsequent steps of a process within the local system. In terms of architecture, the cloud model service interface provides a distinct separation between the lightweight commands executed at the local client and the cloud model implications for reasoning about results. This allows the Gemini CLI to develop an interface to use a cloud-based inference service without having to develop a complete set of intelligence-based resources within the local client itself.

4.4 MCP External Tool Interface

The MCP External Tool Interface is the border of interaction between the Gemini CLI application and the various external tools and services, through the communication of the Model Context Protocol. The concrete architecture of the system uses an interface called the external integration layer, which provides the capabilities of MCP-compatible services to the Gemini CLI application and enables the communication with external tools and services. This is supported by modules such as the MCP client and MCP client manager, which handle communication with MCP-compatible external services. At this border, invocation of tools and their associated execution parameters and context are sent from the system to the external MCP services for execution. The external MCP services execute the tool invocations and return back structured results that the Gemini CLI application can use to continue executing or to generate the final response. The system, therefore, is able to access capabilities that are not locally implemented but can be accessed via a single unified execution flow. From the architectural perspective, it is important to understand that the MCP External Tool Interface provides a consistent method for the system to connect to the external tools/services. A consistent method of connecting to external tools/services makes the architecture of the entire system more extensible, as external tools/services can be integrated without modifying the main orchestration logic of the system.

4.5 Agent Delegation Interface

The Agent Delegation Interface provides the means through which the System interacts with delegated agents or remote agents to perform more intricate tasks. The Task Delegation System provides the architecture's Agent Delegation Interface, which allows the System to distribute work to scoped sub-agents and/or to external Agent endpoints throughout the process of performing tasks. In the concrete architecture, this interface is supported by task scheduling and remote agent coordination mechanisms. The data being transmitted between a System and an agent(s) through this interface would include delegated tasks, sub-task definitions, scoped contexts, execution states that have been generated during the execution of the sub-tasks, and the results of sub-tasks. This enables the System to break complex requests into smaller requests using this interface, and ultimately, to coordinate the execution of these requests that will require more than one direct processing path. From an architect-in-the-loop's perspective, the Agent Delegation Interface allows the System to move away from isolated and local executions. It adds an agent delegation mechanism that fosters more flexible and distributed task processing, therefore making the architecture more capable of addressing multi-step workflows or specialized workflow processing.

4.6 Authentication and Configuration Interface

The Authentication & Configuration Interface is the interface in which the Gemini CLI obtains authentication for its credentials via the input of the configuration state and the access-control context of the system (external services) and to provide for secure communications and protected actions. The bulk of the support for this interface comes from the Configuration System, along with the Security and Execution Environment in the Implementation (concrete architecture). In practice, this includes loading configuration data from configuration files and validating sensitive actions through policy checks before execution. Typically, there are many pieces of data exchanged between these two systems (the input information). The most common types of data include the following: (1) API keys, (2) access tokens, (3) user preferences, (4) configuration files, and (5) other runtime settings. This information enables the system to establish authenticated communications with cloud model services and other external tools, while providing the execution environment with the appropriate operating context required for the execution of those external services. (i.e., the output from the authentication and configuration interface provides the rest of the system with validated configuration and access control context.) From an architectural perspective, the Authentication and Configuration Interface creates a controlled supporting boundary between the internal system and all of the external services. This interface is crucial from a security enforcement standpoint, as it defines how security will be enforced through both authentication and authorization, as well as how configuration will be applied, and how business policies will be considered within the overall architecture.

5 Use Cases

The following section details two common use cases for GeminiCLI - Debug a failing command and refactor codebase. Expanding from Report 1, these sequence diagrams highlight the key components and function calls that take place during execution.

5.1 UC1: Debug a Failing Command

Description
A developer encounters a runtime error when executing a command in their project and asks Gemini CLI to diagnose and fix the issue. Gemini reproduces the failure, inspects relevant files and error output, applies a targeted fix, and validates the solution by re-running the command. As a specific example, the developer enters the prompt, "This script crashes with a null reference error, debug and fix it." This use case shows how Gemini CLI combines reasoning with tool execution and sandboxed validation to safely debug code.

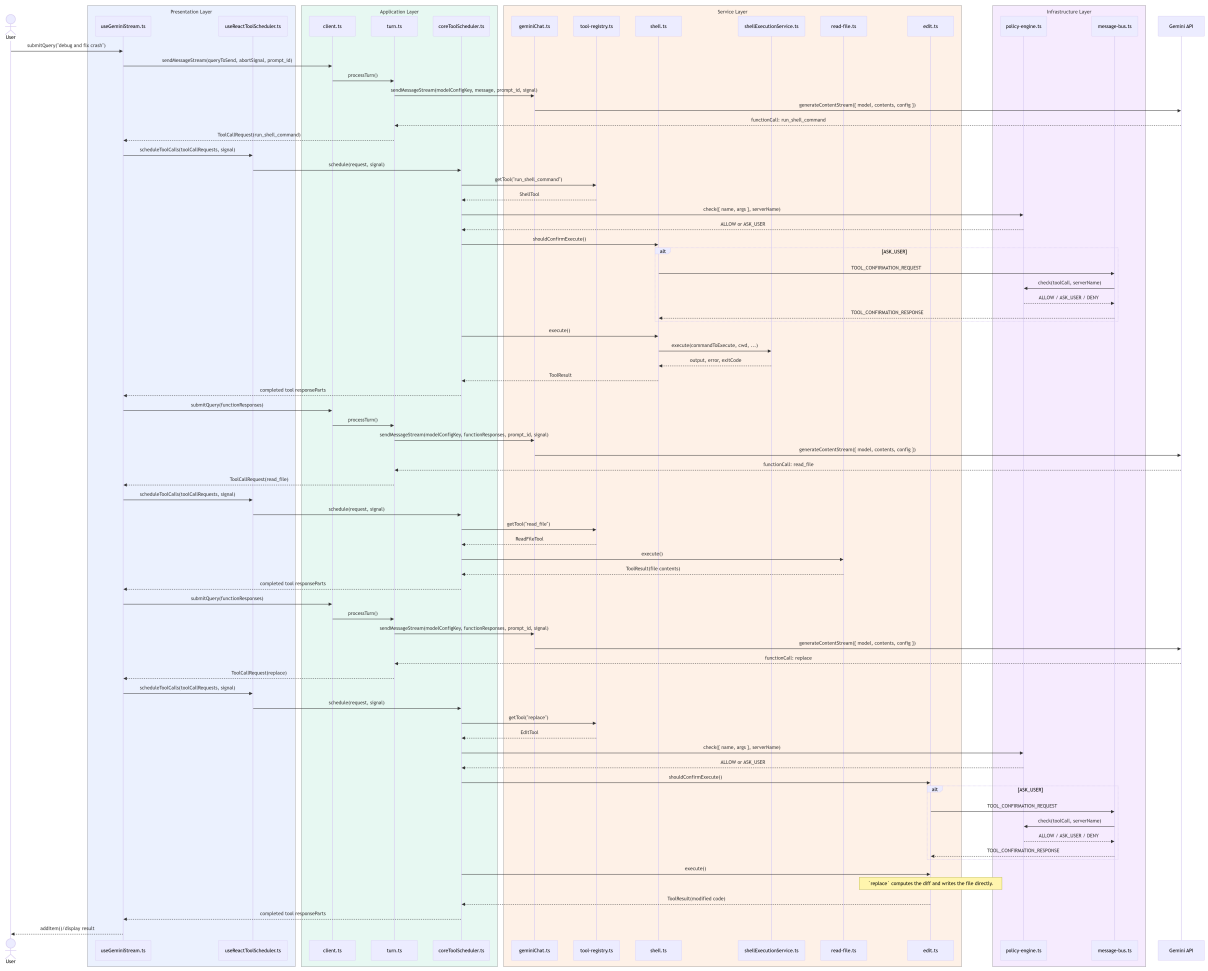

Figure 4: UC1 Sequence Diagram: Debug a Failing Command

Main Flow

1. The user submits the request in useGeminiStream.ts through submitQuery("debug and fix crash").
2. submitQuery() forwards the request to geminiClient.sendMessageStream(...) in useGeminiStream.ts, entering the application flow in client.ts.
3. GeminiClient.sendMessageStream() calls processTurn(), which creates a Turn and delegates to Turn.run() in client.ts and turn.ts.
4. Turn.run() asks GeminiChat.sendMessageStream() to send the prompt and tool declarations to the Gemini API; the actual outbound model call is generateContentStream(...) in geminiChat.ts
5. The model responds with a function call for run_shell_command; Turn.handlePendingFunctionCall() converts that into a ToolCallRequestInfo and emits a tool-call request in turn.ts.
6. The presentation layer receives that event in processGeminiStreamEvents() and sends it to scheduleToolCalls(...) in packages/cli/src/ui/hooks/useGeminiStream.ts.
7. useReactToolScheduler() forwards the request to CoreToolScheduler.schedule(...) in useReactToolScheduler.ts.
8. CoreToolScheduler.__schedule() resolves the tool by calling this.config.getToolRegistry().getTool(reqInfo.name); the registry implementation is ToolRegistry.getTool() in coreToolScheduler.ts and tool-registry.ts.

9. Before execution, CoreToolScheduler validates the action with this.config.getPolicyEngine().check(...) in coreToolScheduler.ts, using PolicyEngine.check() from policy-engine.ts.
10. If confirmation is needed, ShellToolInvocation.shouldConfirmExecute() uses getMessageBusDecision() and emits TOOL_CONFIRMATION_REQUEST through MessageBus.publish() in tools.ts, and message-bus.ts.
11. Once approved, ToolExecutor.execute() runs ShellToolInvocation.execute() in tool-executor.ts and shell.ts.
12. ShellToolInvocation.execute() delegates the actual process run to ShellExecutionService.execute() in shell.ts and shellExecutionService.ts, then returns formatted stderr, output, and exit-code details.
13. The completed shell result is returned to the presentation layer, and handleCompletedTools() resubmits the tool response to the model with submitQuery(functionResponses, { isContinuation: true }, prompt_id) in useGeminiStream.ts.
14. The model next requests read_file; the same scheduler flow resolves ReadFileTool, whose ReadFileToolInvocation.execute() calls processSingleFileContent(...) in read-file.ts.
15. After the file contents are returned and resubmitted, the model requests replace; CoreToolScheduler again resolves the tool, checks policy, and, if needed, routes confirmation through the message bus in coreToolScheduler.ts.
16. ToolExecutor.execute() then runs EditToolInvocation.execute() in.ts. The edit flow generates a diff with Diff.createPatch(...) in getConfirmationDetails() and writes the updated file directly.
17. The final tool result returns to the UI, and addItem() / handleContentEvent() display the debugging summary in useGeminiStream.ts.

5.2 UC2: Refactor Code

Description :
A developer requests a refactoring change on their code base through the CLI, Gemini generates and applies the refactored implementation within the codebase and returns a summary of the results. For example, the developer enters the prompt, " Refactor this function from promise-based .then() chains to modern async/await syntax" . This use case demonstrates how the core components orchestrate a multi-turn conversation with the Gemini API to accomplish this safely.

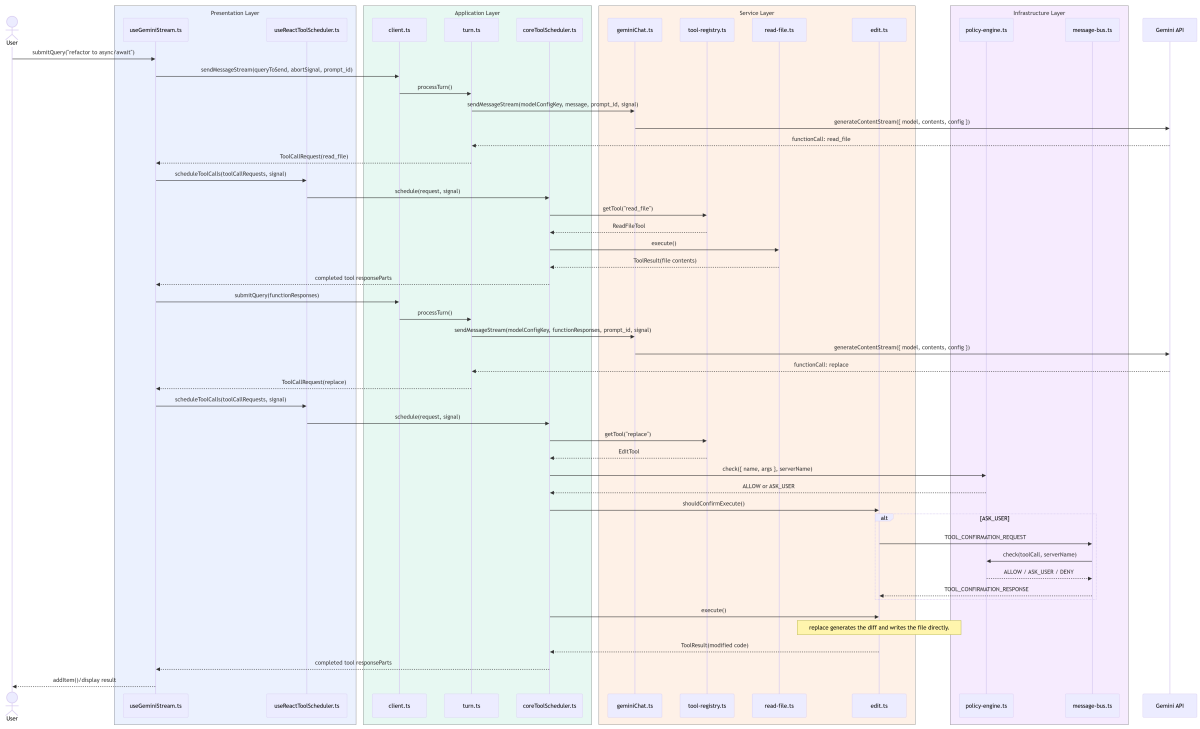

Figure 5: UC2 Sequence Diagram: Refactor Code

Main Flow:

1. The user enters the refactor request in useGeminiStream.ts via submitQuery("refactor to async/await").
2. submitQuery() forwards the request to geminiClient.sendMessageStream(...) in useGeminiStream.ts, which enters GeminiClient.sendMessageStream() in client.ts.
3. GeminiClient.sendMessageStream() invokes processTurn(), which constructs the active turn and delegates to Turn.run() in client.ts and turn.ts.
4. Turn.run() sends the prompt and available tool declarations through GeminiChat.sendMessageStream() to the Gemini API, using generateContentStream(...) in geminiChat.ts and geminiChat.ts.
5. The model first requests read_file; Turn.handlePendingFunctionCall() converts the model function call into a ToolCallRequestInfo in turn.ts.
6. The UI receives that event in processGeminiStreamEvents() and forwards it to scheduleToolCalls(...) in useGeminiStream.ts.
7. useReactToolScheduler() creates and uses CoreToolScheduler.schedule(...) in /useReactToolScheduler.ts.
8. CoreToolScheduler.__schedule() resolves the requested tool through this.config.getToolRegistry().getTool(reqInfo.name), backed by ToolRegistry.getTool() in coreToolScheduler.ts and tool-registry.ts.
9. ReadFileToolInvocation.execute() reads the source file by calling processSingleFileContent(...) in read-file.ts, and the result is returned to the UI.
10. handleCompletedTools() packages the tool response and resubmits it to Gemini with submitQuery(functionResponses, { isContinuation: true }, prompt_id) in useGeminiStream.ts.
11. The model then requests replace to perform the refactor; Turn.handlePendingFunctionCall() emits that tool call in turn.ts.

12. CoreToolScheduler._schedule() resolves replace, then validates the modification through this.config.getPolicyEngine().check(...) in coreToolScheduler.ts, using PolicyEngine.check() in policy-engine.ts.
13. If confirmation is required, EditToolInvocation.shouldConfirmExecute() triggers TOOL_CONFIRMATION_REQUEST through getMessageBusDecision() and MessageBus.publish() in tools.ts and message-bus.ts.
14. After approval, ToolExecutor.execute() invokes EditToolInvocation.execute() in tool-executor.ts and edit.ts.
15. The edit tool computes the patch context with Diff.createPatch(...) in getConfirmationDetails() and then writes the refactored implementation directly from EditToolInvocation.execute() in edit.ts
16. The completed edit result is returned to the UI, where addItem() displays the final refactor outcome in useGeminiStream.ts.

6 Data Dictionary

- **Terminal Interface:** [Type: Subsystem] The Presentation Layer component acting as the user-facing boundary, responsible for input capture and output rendering using a dynamic, terminal-based DOM with React and Ink.
- **External Integration Layer:** [Type: Subsystem] The Service Layer subsystem responsible for discovering and connecting the local CLI to external tools by utilizing the MCP.
- **React:** [Type: Framework/Library] A declarative UI framework used to structure the presentation layer's terminal-based DOM.
- **Ink:** [Type: Framework/Library] A React-based library used within the Terminal Interface to render a dynamic, component-based terminal DOM.
- **Monorepo:** [Type: Development Concept] The version control strategy used to house both the packages/cli and packages/core directories within a single repository.
- **Orchestration Engine:** [Type: Subsystem] The central application controller within the Application Layer that handles session management, LLM network communication, and event-driven execution scheduling.
- **Policy Engine:** [Type: Component] A safety mechanism within the Security & Execution Environment that elevates requested tool invocations against strict safety rules.
- **Tool Execution Framework:** [Type: Subsystem] A Service Layer subsystem built on a registry pattern that dynamically loads and executes local tools, such as shell execution and file system operations.
- **Tool Registry:** [Type: Component] A dynamic directory managed within the Tool Execution Framework that tracks available tools and maps tool names to actual tool implementations.
- **Security & Execution Environment:** [Type: Subsystem] An Infrastructure Layer component isolated to strictly enforce safety rules and validate state-modifying tool executions.
- **Task Delegation System:** [Type: Subsystem] A Service Layer component that oversees multi-step operations by managing scoped sub-agents and enabling A2A communication directly over HTTP.
- **Understand:** [Type: Tool] A static analysis tool used to examine module dependencies and derive the system's concrete architecture directly from the repository.

7 Naming Conventions

- **.ts / .tsx:** File extensions representing TypeScript and TypeScript React (JSX) files, which house the concrete logic and interface implementations analyzed in the report (ex. useGeminiStream.ts,

AppContainer.tsx).

- **CLI (Command Line Interface):** In this architecture, specifically refers to the user-facing client (packages/cli) rather than the entire system.
- **A2A (Agent-to-Agent):** An acronym used to denote remote agent instances and servers that coordinate with the local CLI to handle distributed execution of complex tasks.
- **MCP (Model Context Protocol):** The standard protocol identifier used when bridging the Gemini CLI to external tools, treating external services as native tools within the registry.
- **packages/:** The directory prefix used within the monorepo to separate physical subsystems, heavily referenced when mapping the conceptual architecture to concrete components like packages/cli and packages/core.

8 Conclusions

This report presented a comprehensive concrete architectural analysis of the Gemini CLI, transitioning from the conceptual model proposed in Assignment 1 to the physical reality of the system's implemented monorepo. By utilizing the Understand static analysis tool to extract module dependencies, the analysis revealed that the actual codebase is significantly more granular than initially modeled, expanding the conceptual three-tier design into a four-tier layered architecture encompassing seven distinct components. A major finding of this reflexion analysis was the strict physical decoupling required to satisfy the system's operational and security constraints. The conceptual "Core" subsystem was fundamentally fragmented into the Orchestration Engine, Security and Execution Environment, and Configuration System. This separation of concerns ensures that safety policies can independently evaluate and veto tool execution without being bypassed by orchestration routing logic, while allowing configuration data to remain globally accessible across the application.

Similarly, the conceptual "Tools" subsystem was physically divided into the Tool Execution Framework, External Integration Layer, and Task Delegation System to handle the distinct lifecycles and network boundaries required by local operations, external MCP servers, and distributed A2A HTTP instances. Despite this necessary physical fragmentation, the concrete architecture successfully preserved the system's foundational boundaries and prevented architectural decay. The Presentation Layer maintains a strict unidirectional dependency flow, interacting solely with the Application Layer and never bypassing it to access lower-level Execution or Infrastructure layers. Our second-level analysis of the Tool Execution Framework highlighted the intricate internal collaboration that spans tool discovery, scheduling, policy enforcement, and execution. Ultimately, this analysis demonstrates how the Gemini CLI's granular physical decomposition effectively balances system extensibility with secure, isolated task delegation at the code level. By enforcing strict physical boundaries and structurally isolating critical concerns like security and configuration, the implemented architecture successfully prevents structural degradation while providing a robust foundation for orchestrating complex, real-world developer workflows.

9 Lessons Learned

Through analyzing the concrete architecture of the Gemini CLI, our team gained a deeper understanding of how conceptual models translate into physical implementations. A key takeaway was the value of utilizing the Understand tool to examine module dependencies across the monorepo. This process revealed that the concrete architecture is significantly more granular than our initial three-tier conceptual model, expanding into seven major components across four distinct layers. One of the most surprising realizations was how monorepos often require strict fragmentation in practice to satisfy differing operational requirements. While we initially designed a monolithic 'Core' in our A1 model, analyzing the concrete code taught us that security policies and global configurations naturally force a physical decoupling to prevent routing bypasses. This highlighted the practical difference between logical grouping and physical implementation. Additionally, conducting our second-level reflexion analysis on the Tool Execution Framework exposed the hidden and intrinsic complexity required to connect AI-driven reasoning with local developer environments. We learned

that executing a simple system tool isn't just a direct function call, instead it requires an intricate, multi-stage pipeline involving tool discovery, scheduling, strict policy enforcement, and confirmation management. Ultimately, this work demonstrated that preventing architectural decay in a production system requires rigorous structural discipline at the code level.

10 AI Collaboration Report

Our group selected ChatGPT as our mandatory virtual AI teammate because its conversational interface and strong natural language generation capabilities made it highly suitable for revising technical writing and organizing structural content. Our primary interaction protocol with the AI involved a human-in-the-loop iterative prompting strategy. We did not use the AI to establish the architecture or perform the static analysis. Instead, human team members used the Understand tool to extract the dependency graph and define the physical subsystems. Once the technical facts were established and a rough draft was written, we prompted ChatGPT by supplying the raw text alongside specific constraints (e.g., "reduce redundancy," "improve sentence flow," or "format these descriptions into a clear summary").

This strategy required strict quality control from the team. Since we know LLMs can hallucinate architectural details, every output generated by ChatGPT was manually reviewed against the original Understand dependency graphs by at least two human team members before being integrated into the report. No architectural claims were accepted without verifiable grounding in the source code. The human-AI team dynamics were highly collaborative but firmly human-led; the AI acted strictly as a copyeditor and formatter rather than a decision-maker. Quantitatively, we estimate that the human team members generated 100% of the technical architectural facts and roughly 75% of the foundational text, while the AI teammate materially shaped, revised, or formatted the remaining 25% of the raw text to improve readability and flow.

References

[1] Google AI, "Gemini API Documentation," https://ai.google.dev/gemini-api/docs/gemini-cli.

[2] Vadim Demedes, "Ink Documentation," https://github.com/vadimdemedes/ink#why-ink.

[3] Meta, "Thinking in React," https://react.dev/learn/thinking-in-react.

[4] OpenJS Foundation, "The Node.js Event Loop," https://nodejs.org/en/learn/asynchronous-work/event-loop.

[5] John Alateras, "Unpacking the Gemini CLI: A High-Level Architectural Overview," Medium, https://medium.com/@jalateras/unpacking-the-gemini-cli-a-high-level-architectural-overview-99212f6780e7.

[6] Cezar Milo, "Gemini CLI Project Architecture Analysis," Dev.to, https://dev.to/czmilo/gemini-cli-project-architecture-analysis-3onn.

[7] Group Report, "A1 Report: Gemini CLI Conceptual Architecture," course project submission, 2026.

[8] Google Gemini CLI source, tool-registry.ts.

[9] Google Gemini CLI source, coreToolScheduler.ts.

[10] Google Gemini CLI source, tool-executor.ts.

[11] Google Gemini CLI source, shell.ts.

[12] Google Gemini CLI source, shellExecutionService.ts.

[13] Google Gemini CLI source, policy-engine.ts.

[14] Google Gemini CLI source, message-bus.ts.

[15] CISC/CMPE 322 course material, "Reflexion Models."

[16] CISC/CMPE 322 course material, "Architecture Styles."

[17] Queen's University onQ, "Reference Architecture," https://onq.queensu.ca/d2l/le/content/1132545/viewContent/6711621/